CHAPTER 7

Scope of Architecture Characteristics

A prevailing axiomatic assumption in the software architecture world had traditionally placed the scope of architecture characteristics at the system level. For example, when architects talk about scalability, they generally couch that discussion around the scalability of the entire system. That was a safe assumption a decade ago, when virtually all systems were monolithic. With the advent of modern engineering techniques and the architecture styles they enabled, such as microservices, the scope of architecture characteristics has narrowed considerably. This is a prime example of an axiom slowly becoming outdated as the software development ecosystem continues its relentless evolution.

During the writing of the *Building Evolutionary Architectures* book, the authors needed a technique to measure the structural evolvability of particular architecture styles. None of the existing measures offered the correct level of detail. In "Structural Measures" on page 79, we discuss a variety of code-level metrics that allow architects to analyze structural aspects of an architecture. However, all these metrics only reveal low-level details about the code, and cannot evaluate dependent components (such as databases) outside the code base that still impact many architecture characteristics, especially operational ones. For example, no matter how much an architect puts effort into designing a performant or elastic code base, if the system uses a database that doesn't match those characteristics, the application won't be successful.

When evaluating many operational architecture characteristics, an architect must consider dependent components outside the code base that will impact those characteristics. Thus, architects need another method to measure these kinds of dependencies. That lead the *Building Evolutionary Architectures* authors to define the term *architecture quantum*. To understand the architecture quantum definition, we must preview one key metric here, connascence.

Coupling and Connascence

Many of the code-level coupling metrics, such as *afferent* and *efferent* coupling (described in "Structural Measures" on page 79), reveal details at a too fine-grained level for architectural analysis. In 1996, Meilir Page-Jones published a book titled *What Every Programmer Should Know About Object Oriented Design* (Dorset House) that included several new measures of coupling he named *connascence*, which is defined as follows:

Connascence
: Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system

He defined two types of connascence: *static*, discoverable via static code analysis, and *dynamic*, concerning runtime behavior. To define the architecture quantum, we needed a measure of how components are "wired" together, which corresponds to the connascence concept. For example, if two services in a microservices architecture share the same class definition of some class, like *address*, we say they are *statically* connascent with each other—changing the shared class requires changes to both services.

For dynamic connascence, we define two types: *synchronous* and *asynchronous*. Synchronous calls between two distributed services have the caller wait for the response from the callee. On the other hand, *asynchronous* calls allow fire-and-forget semantics in event-driven architectures, allowing two different services to differ in operational architecture

Architectural Quanta and Granularity

Component-level coupling isn't the only thing that binds software together. Many business concepts semantically bind parts of the system together, creating *functional cohesion*. To successfully design, analyze, and evolve software, developers must consider all the coupling points that could break.

Many science-literate developers know of the concept of quantum from physics, the minimum amount of any physical entity involved in an interaction. The word quantum derives from Latin, meaning "how great" or "how much." We have adopted this notion to define an *architecture quantum*:

Architecture quantum
: An independently deployable artifact with high functional cohesion and synchronous connascence

This definition contains several parts, dissected here:

Independently deployable

An architecture quantum includes all the necessary components to function independently from other parts of the architecture. For example, if an application uses a database, it is part of the quantum because the system won't function without it. This requirement means that virtually all legacy systems deployed using a single database by definition form a quantum of one. However, in the microservices architecture style, each service includes its own database (part of the *bounded context* driving philosophy in microservices, described in detail in Chapter 17), creating multiple quanta within that architecture.

High functional cohesion

Cohesion in component design refers to how well the contained code is unified in purpose. For example, a `Customer` component with properties and methods all pertaining to a *Customer* entity exhibits high cohesion; whereas a `Utility` component with a random collection of miscellaneous methods would not.

High functional cohesion implies that an architecture quantum does something purposeful. This distinction matters little in traditional monolithic applications with a single database. However, in microservices architectures, developers typically design each service to match a single workflow (a *bounded context*, as described in "Domain-Driven Design's Bounded Context" on page 94), thus exhibiting high functional cohesion.

Synchronous connascence

Synchronous connascence implies synchronous calls within an application context or between distributed services that form this architecture quantum. For example, if one service in a microservices architecture calls another one synchronously, each service cannot exhibit extreme differences in operational architecture characteristics. If the caller is much more scalable than the callee, timeouts and other reliability concerns will occur. Thus, synchronous calls create dynamic connascence for the length of the call—if one is waiting for the other, their operational architecture characteristics must be the same for the duration of the call.

Back in Chapter 6, we defined the relationship between traditional coupling metrics and connascence, which didn't include our new *communication connascence* measure. We update this diagram in Figure 7-1.

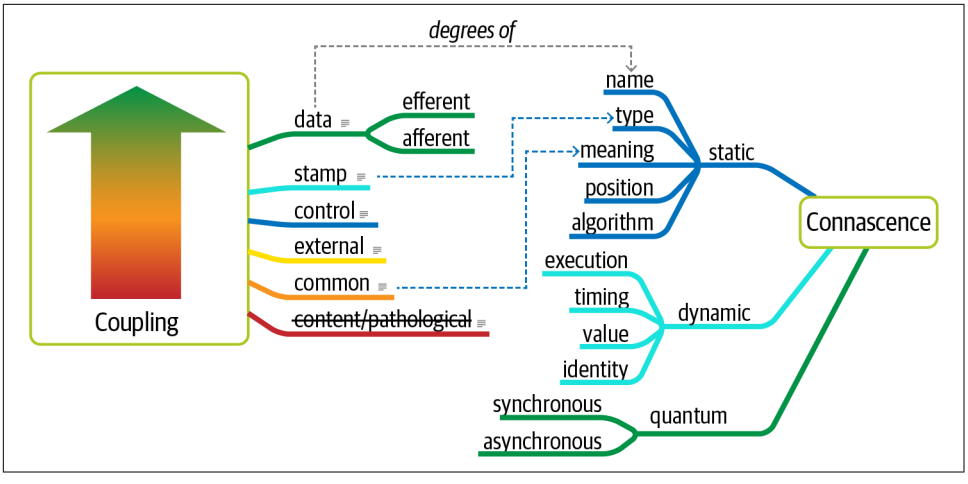

Figure 7-1. Adding quantum connascence to the unified diagram

For another example, consider a microservices architecture with a `Payment` service and an `Auction` service. When an auction ends, the `Auction` service sends payment information to the `Payment` service. However, let's say that the payment service can only handle a payment every 500 ms—what happens when a large number of auctions end at once? A poorly designed architecture would allow the first call to go through and allow the others to time out. Alternatively, an architect might design an asynchronous communication link between `Payment` and `Auction`, allowing the message queue to temporarily buffer differences. In this case, asynchronous connascence creates a more flexible architecture. We cover this subject in great detail in Chapter 14.

Domain-Driven Design's Bounded Context

Eric Evans' book *Domain-Driven Design* (Addison-Wesley Professional) has deeply influenced modern architectural thinking. *Domain-driven design* (DDD) is a modeling technique that allows for organized decomposition of complex problem domains. DDD defines the *bounded context*, where everything related to the domain is visible internally but opaque to other bounded contexts. Before DDD, developers sought holistic reuse across common entities within the organization. Yet creating common shared artifacts causes a host of problems, such as coupling, more difficult coordination, and increased complexity. The *bounded context* concept recognizes that each entity works best within a localized context. Thus, instead of creating a unified `Customer` class across the entire organization, each problem domain can create its own and reconcile differences at integration points.

The architecture quantum concept provides the new scope for architecture characteristics. In modern systems, architects define architecture characteristics at the quantum level rather than system level. By looking at a narrower scope for important operational concerns, architects may identify architectural challenges early, leading to hybrid architectures. To illustrate scoping provided by the architecture quantum measure, consider another architecture kata, *Going, Going, Gone*.

Case Study: Going, Going, Gone

In Chapter 5, we introduced the concept of an architecture kata. Consider this one, concerning an online auction company. Here is the description of the architecture kata:

Description

An auction company wants to take its auctions online to a nationwide scale. Customers choose the auction to participate in, wait until the auction begins, then bid as if they are there in the room with the auctioneer.

Users

Scale up to hundreds of participants per auction, potentially up to thousands of participants, and as many simultaneous auctions as possible.

Requirements

- Auctions must be as real-time as possible.
- Bidders register with a credit card; the system automatically charges the card if the bidder wins.
- Participants must be tracked via a reputation index.
- Bidders can see a live video stream of the auction and all bids as they occur.
- Both online and live bids must be received in the order in which they are placed.

Additional context

- Auction company is expanding aggressively by merging with smaller competitors.
- Budget is not constrained. This is a strategic direction.
- Company just exited a lawsuit where it settled a suit alleging fraud.

Just as in "Case Study: Silicon Sandwiches" on page 69, an architect must consider each of these requirements to ascertain architecture characteristics:

1. "Nationwide scale," "scale up to hundreds of participants per auction, potentially up to thousands of participants, and as many simultaneous auctions as possible," "auctions must be as real-time as possible."

Each of these requirements implies both scalability to support the sheer number of users and elasticity to support the bursty nature of auctions. While the requirements explicitly call out scalability, elasticity represents an implicit characteristics based on the problem domain. When considering auctions, do users all politely spread themselves out during the course of bidding, or do they become more frantic near the end? Domain knowledge is crucial for architects to pick up implicit architecture characteristics. Given the real-time nature of auctions, an architect will certainly consider performance a key architecture characteristic.

2. "Bidders register with a credit card; the system automatically charges the card if the bidder wins," "company just exited a lawsuit where it settled a suit alleging fraud."

   Both these requirements clearly point to security as an architecture characteristic. As covered in Chapter 5, security is an implicit architecture characteristic in virtually every application. Thus, architects rely on the second part of the definition of architecture characteristics, that they influence some structural aspect of the design. Should an architect design something special to accommodate security, or will general design and coding hygiene suffice? Architects have developed techniques for handling credit cards safely via design without necessarily building special structure. For example, as long as developers make sure not to store credit card numbers in plain text, to encrypt while in transit, and so on, then the architect shouldn't have to build special considerations for security.

   However, the second phrase should make an architect pause and ask for further clarification. Clearly, some aspect of security (fraud) was a problem in the past, thus the architect should ask for further input no matter what level of security they design.

3. "Participants must be tracked via a reputation index."

   This requirement suggests some fanciful names such as "anti-trollability," but the *track* part of the requirement might suggest some architecture characteristics such as auditability and loggability. The deciding factor again goes back to the defining characteristic—is this outside the scope of the problem domain? Architects must remember that the analysis to yield architecture characteristics represents only a small part of the overall effort to design and implement an application—a lot of design work happens past this phase! During this part of architecture definition, architects look for requirements with structural impact not already covered by the domain.

   Here's a useful litmus test architects use to make the determination between domain versus architecture characteristics is: does it require domain knowledge to implement, or is it an abstract architecture characteristic? In the Going, Going, Gone kata, an architect upon encountering the phrase "reputation index" would

seek out a business analyst or other subject matter expert to explain what they had in mind. In other words, the phrase "reputation index" isn't a standard definition like more common architecture characteristics. As a counter example, when architects discuss *elasticity*, the ability to handle bursts of users, they can talk about the architecture characteristic purely in the abstract—it doesn't matter what kind of application they consider: banking, catalog site, streaming video, and so on. Architects must determine whether a requirement isn't already encompassed by the domain *and* requires particular structure, which elevates a consideration to architecture characteristic.

4. "Auction company is expanding aggressively by merging with smaller competitors."

   While this requirement may not have an immediate impact on application design, it might become the determining factor in a trade-off between several options. For example, architects must often choose details such as communication protocols for integration architecture: if integration with newly merged companies isn't a concern, it frees the architect to choose something highly specific to the problem. On the other hand, an architect may choose something that's less than perfect to accommodate some additional trade-off, such as interoperability. Subtle implicit architecture characteristics such as this pervade architecture, illustrating why doing the job well presents challenges.

5. "Budget is not constrained. This is a strategic direction."

   Some architecture katas impose budget restrictions on the solution to represent a common real-world trade-off. However, in the Going, Going, Gone kata, it does not. This allows the architect to choose more elaborate and/or special-purpose architectures, which will be beneficial given the next requirements.

6. "Bidders can see a live video stream of the auction and all bids as they occur," "both online and live bids must be received in the order in which they are placed."

   This requirement presents an interesting architectural challenge, definitely impacting the structure of the application and exposing the futility of treating architecture characteristics as a system-wide evaluation. Consider availability—is that need uniform throughout the architecture? In other words, is the availability of the one bidder more important than availability for one of the hundreds of bidders? Obviously, the architect desires good measures for both, but one is clearly more critical: if the auctioneer cannot access the site, online bids cannot occur for anyone. Reliability commonly appears with availability; it addresses operational aspects such as uptime, as well as data integrity and other measures of how reliable an application is. For example, in an auction site, the architect must ensure that the message ordering is reliably correct, eliminating race conditions and other problems.

This last requirement in the Going, Going, Gone kata highlights the need for a more granular scope in architecture than the system level. Using the architecture quantum measure, architects scope architecture characteristics at the quantum level. For example, in Going, Going, Gone, an architect would notice that different parts of this architecture need different characteristics: streaming bids, online bidders, and the auctioneer are three obvious choices. Architects use the architecture quantum measure as a way to think about deployment, coupling, where data should reside, and communication styles within architectures. In this kata, an architect can analyze the differing architecture characteristics per architecture quantum, leading to hybrid architecture design earlier in the process.

Thus, for Going, Going, Gone, we identified the following quanta and corresponding architecture characteristics:

Bidder feedback
: Encompasses the bid stream and video stream of bids

- Availability
- Scalability
- Performance

Auctioneer
: The live auctioneer

- Availability
- Reliability
- Scalability
- Elasticity
- Performance
- Security

Bidder
: Online bidders and bidding

- Reliability
- Availability
- Scalability
- Elasticity